PRODUCT AND TECHNICAL OVERVIEW

Sanvicasa Invoice & Billing

From a single-company tool to a secure, mobile-ready subscription platform

October 2026

Confidential. Prepared for client discussion.

SUMMARY

What this document covers

Sanvicasa Invoice & Billing replaces a manual, spreadsheet-based purchase tracker. Staff photograph or upload a supplier invoice, the system reads it with AI, a person checks it, and spending and payments appear on live dashboards.

This document explains the product as it works today, the technical changes we are making, and why. The aim is a platform that works on the web and on phones, can serve several companies on subscription plans, stays fast as it grows, and keeps each customer's data separate and protected.

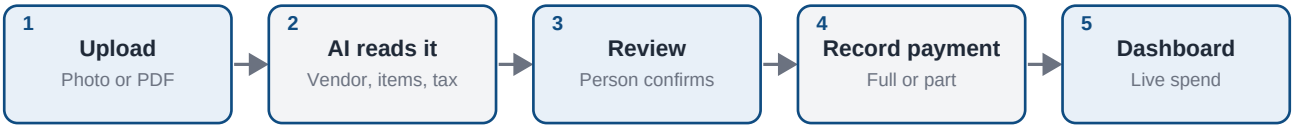
Goal	What it means for you
Mobile apps	Android and iPhone apps so site staff can photograph invoices on the spot.
Subscription plans	Offer the product to several companies, each on its own plan with clear limits.
Control of AI cost	Monthly page limits per project and for the whole company, so AI usage never surprises you.
A management console	A separate admin area for our team to manage customers, plans, support and logs.
Security and reliability	Company data kept separate, secrets encrypted, every important action recorded, safe backups.
Speed	Fast pages and smooth uploads, even with many invoices and many users.

In one sentence: same product your team already uses, rebuilt on a foundation that supports phones, multiple companies, paid plans and stronger security.

THE PRODUCT TODAY

How it works for your team

The daily workflow is deliberately short. Most of the effort that used to go into typing invoices into a spreadsheet is now automatic.



What each person can do

Role	What they do
Staff	Upload invoices, review the AI's reading, record payments on their assigned projects.
Admin	Everything staff can do, plus dashboards, projects, vendors, categories, users, exports and settings.

Key features

- **AI invoice reading** using Azure Document Intelligence: vendor, invoice number, dates, line items and taxes.
- **Review before saving**: nothing is final until a person confirms it, and edits to AI fields are logged.
- **Duplicate protection**: the same invoice number for the same vendor cannot be saved twice, and identical files are caught before any AI cost.
- **Projects and teams**: spending is tracked per site, and staff only see the projects they are assigned to.
- **Payments**: full and partial payments with remaining balance tracking.
- **Dashboards and exports**: spend by category, vendor, project and month, with Excel and JSON export.
- **Activity and error logs** for accountability and support.

TECHNOLOGY

What it is built on, and why we are changing it

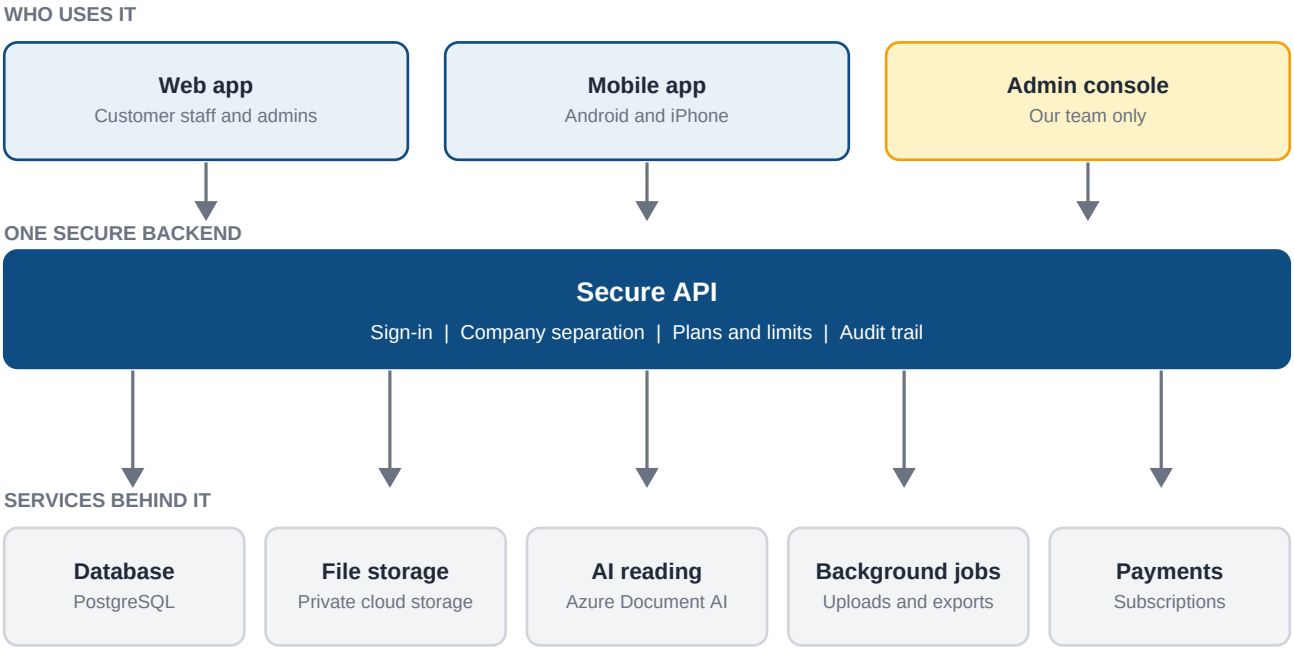
Today the product is a single web application: the screens and the server logic live together, using one database for one company. That is a good, fast way to prove a product. It has limits we now need to move past.

Area	Today	Limit we are removing
App type	One web application (Next.js)	A phone app cannot use server pages directly. It needs a proper API.
Customers	One company	No separation between companies, so it cannot be sold to several customers safely.
Plans and billing	None	No subscription tiers, trial, or usage limits tied to a plan.
Management	Admin screens inside the same app	Our team has no separate console for support, billing and logs.
AI usage	Shared key, no limits	AI cost cannot be controlled or attributed to a project.
Database	PostgreSQL (free tier today)	Short restore window and sleep-on-idle are not suitable for paying customers.

What stays the same

We are not throwing the product away. The data model, the review workflow, the permission rules and the AI reading are kept. The main changes are structural: separating the web app, the admin console, the mobile app and the backend, so each can grow independently.

The new structure



WHAT WE ARE ADDING

Capabilities and the value of each

Capability	How it works	Benefit
Mobile apps (Android and iPhone)	One shared app for both phones. Photos are shrunk on the phone and sent straight to secure storage.	Staff capture invoices on site; faster and fewer lost paper bills.
Multiple companies (multi-tenant)	Every record belongs to a company. The database itself refuses to show one company's data to another.	One platform serves many customers safely.
Subscription plans (one project each)	Each plan includes one project. A company that needs more projects buys the plan again, one per extra project. Plans also set limits on users and monthly AI pages. Payment events from the payment provider update the plan automatically.	Simple, predictable pricing that grows with each project; free trials; clean upgrades.
Email support with ticket numbers	A help page prefilled with the company, plan and error details, plus an optional form that logs a ticket and confirms it by email. Replies stay in email.	Faster answers and fewer back-and-forth questions.
AI page limits	Pages are counted per project and company-wide each month. Deleting an invoice does not give pages back.	Predictable AI cost; usage visible to admins.
Project-level AI keys	Each project can use its own Azure endpoint and key, stored encrypted and never displayed again.	Cost and access separated per project.
Admin console	A separate site for our team, with its own login and mandatory second factor.	Manage customers, plans, support, billing events and logs without database access.
Audit trail	Who did what, to which record, and when, kept permanently and not editable.	Accountability and easier support.

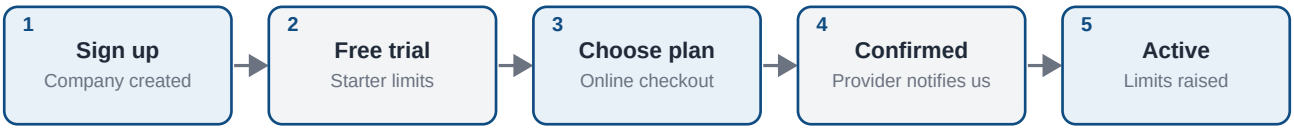
HOW IT FLOWS

Three journeys explained

1. Uploading an invoice (with limits and security)

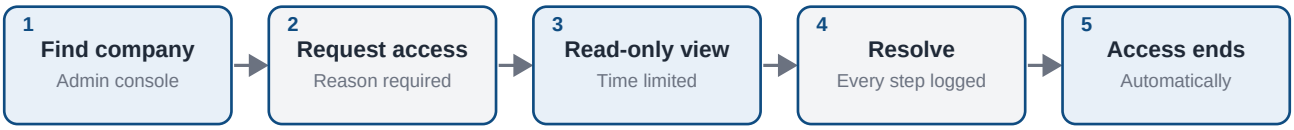
- The same file is detected before any AI cost is spent.
- Pages are reserved before the AI runs. If the system fails, they are refunded automatically.
- Files are kept in private storage and opened only through short-lived links.

2. Joining and upgrading a plan



- Access changes only when the payment provider confirms payment, never just because a page was opened.
- If a payment fails there is a grace period, then read-only access. Data is never deleted for non-payment.

3. Our team supporting a customer



SECURITY

How data is protected

Area	Approach
Company separation	Every record carries a company identifier. Enforced in the application and by the database, with automated tests that try to read another company's data.
Sign-in	Short-lived access tokens with renewable sessions. Phones store them in the device's secure storage. Optional fingerprint or face unlock.
Second factor	Mandatory for our admin console, optional for customers' owners.
Secrets	Customer Azure keys are encrypted before storage and are never shown again. The encryption key is kept outside the database.
Files	Stored privately. Opened only through links that expire within minutes.
Abuse protection	Limits on login attempts and sensitive actions.
Audit	Important actions are recorded permanently with who, what and when.
Backups	Automated, with a long restore window. Restores are rehearsed, not assumed.
Privacy	Per-company data export and deletion on request.

Principle: security is built into how the system is structured, not added on afterwards. Where we say an item is planned, it is not yet live.

WHITE-LABEL AND APP STORES

Branded apps for each client

"White-label" means each client sees the product under their own name, logo and colours. There are three ways to do this. They differ in effort, cost and, importantly, in how the app stores treat them.

Model	How it works	Best for	Watch out for
A. One shared app, branded at sign-in (recommended start)	A single app in our store account. After the user signs in, the app loads that company's name, logo and colours from the server.	Most customers. Fastest to launch; one review and one release for everyone.	The store listing and home-screen icon carry our product name, not the client's.
B. A separate branded app per client, published under our accounts	Same code, built many times with each client's name, icon and identifiers. We publish all of them.	A few clients who need their own icon and name but accept us as publisher.	Stores can treat near-identical apps as repetitive or spam. Every app needs its own listing, review and update.
C. A separate branded app per client, published under the client's own accounts	Same build process, but the client owns the store accounts and we upload on their behalf with delegated access.	Larger clients who require their own store presence and ownership.	Client pays the store fees and needs a company identity number. Slower start-up per client.

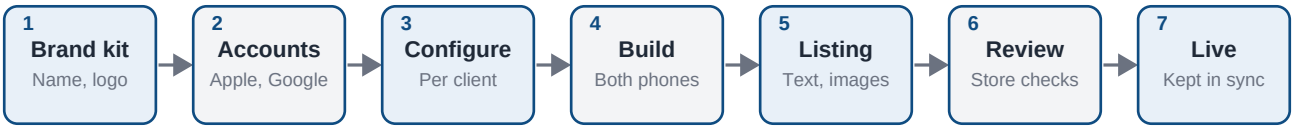
What the stores say

- **Apple:** apps made from a commercial template or app-generation service were banned in 2017. The rule was then revised so such apps are accepted when submitted directly by the provider of the app's content. A separate anti-spam rule (4.3) is also used against near-duplicate apps, and developers have been asked to merge such apps into one container app.
- **Google Play:** there is no white-label rule as such, but the repetitive content policy lists several apps with highly similar functionality and experience as a violation. An informal community answer says one account or the client's own account makes little difference; the similarity of the apps is what matters.
- These sources date from 2017 to 2023 and some are informal. Store rules change, so we re-check the current wording before each submission.

Our recommendation: start with Model A. Offer Model C to clients who insist on their own branded app, because the client is then the publisher of their own content, which is the case the stores accept. Use Model B only for a small number of clients, and only after checking current store rules.

WHITE-LABEL AND APP STORES

How a branded app goes live



What we need from each client for a branded app

Item	Details
Brand kit	App name, logo, app icon (1024 x 1024), brand colours, splash image, and store graphics (Google needs a 1024 x 500 banner).
App identity	A unique app identifier for each store (for example com.clientname.invoices). It cannot change after publishing.
Store accounts	Model C: the client's Apple Developer account and Google Play account, with our team invited by role. Model B: our accounts.
Company verification	Organisation accounts need a company identity number (D-U-N-S). Allow time for verification.
Web addresses	A public privacy policy page and a support page or email for the store listing.
Store forms	Google's data-safety form, Apple's privacy details, content rating, and an in-app account-deletion route.
Reviewer access	A working demo login so the store's reviewer can see the app. A missing demo login is a common reason for rejection.
Push notifications	Separate notification credentials for each branded app (Apple and Google keys).
Support and web address	Optional: the client's own web address (for example invoices.clientname.com) and an email sender in their domain.

How it is built

- **One codebase, many brands.** The mobile app reads its name, identifiers and artwork from a per-client configuration (the standard variant pattern in Expo), so a new brand is a configuration plus assets, not new code.
- **Branding stored on the server.** Each company has brand settings (display name, logo, colours, support contact) that the web and mobile apps load at run time. Changing a logo does not need a store release.
- **Admin console control.** Our team manages each client's branding, plan, and the status of their store listings in one place.
- **Releases multiply.** Every app update must be built, tested and submitted once per branded app. That is why we keep the number of separately branded apps small and automate the build for each.

Indicative effort and cost per branded app

Item	Model A	Model B	Model C
Store accounts	Ours, already needed	Ours, already needed	Client pays (about USD 99 per year Apple, about USD 25 once Google)
Setup per client	Branding settings only	Config, assets, listing, review	Config, assets, listing, review, account access
Each new release	Once	Once per app	Once per app
Store policy risk	Lowest	Highest	Lower

Fees are the stores' published prices at the time of writing and may change.

ROADMAP

Where we are and what comes next

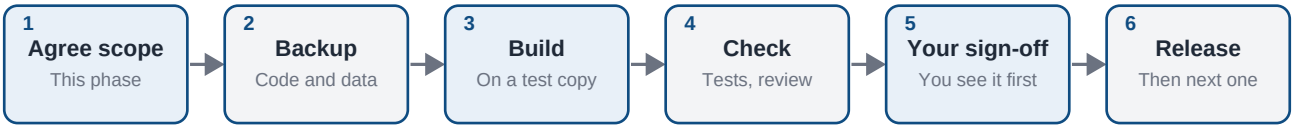
Each phase can be released on its own, so the product keeps working throughout. Durations are confirmed after a short scoping workshop; we do not promise dates before the plan design is settled.

Phase	What it delivers	Status
0. Safety first	Backup and restore routine, secrets cleanup, version control, staging environment.	Partly done. Secrets cleanup done; backup routine and test environment to set up.
AI page limits and project keys	Monthly limits per project and company, encrypted per-project Azure keys, duplicate-file protection.	Built, awaiting testing and deployment
1. Project structure	Web app, shared database package and (next) API, admin console and mobile app as separate parts.	Built, awaiting testing and deployment
2. Companies	Company model, memberships, separated data, database-level protection.	Planned
3. API	Secure backend for all clients, with tokens, rate limits and a documented interface.	Planned
4. Faster uploads and dashboards	Background processing, quicker dashboards, search and pagination.	Planned
5. Plans, limits and support	Plans with one project each, extra projects by buying the plan again, limits on users and AI pages, usage meters, payment provider, and email support with ticket numbers.	Built, awaiting testing and deployment (payments not included)
Accounts and sign-up	Self-service sign-up, email confirmation, invitations, password reset, account deletion, sign-in protections, revocable sessions, company switcher. (Optional second factor is still planned.)	Built, awaiting testing and deployment
6. Admin console	Customers, plans, support access, billing events, logs, feature switches.	Planned
8. Mobile apps and white-label	Android and iPhone apps on the same API; company branding; optional separately branded apps per client.	Planned
9. Launch readiness	Load tests, alerts, recovery drill, legal pages, public store listing.	Planned

HOW WE WORK

Phase by phase, with checks in between

We change the product in small, separate phases rather than one large rebuild. The current system keeps working while each phase is built, checked and approved.



- **Backup before every phase.** A copy of the code and of the database is taken first, and restoring it is tested, not assumed.
- **Test copy first.** New work is tried on a separate test copy of the system, never directly on live data.
- **Safe database changes.** Changes only add things at first, existing data is migrated in a separate step, and old structures are removed last.
- **Automatic checks.** Type checks, linting and a production build run on every phase. Tests that try to read one company's data from another must pass before release.
- **Your review.** Each phase ends with a demonstration and your approval before it is released.
- **Switches for risky features.** New features can be turned on for one company first and turned off again quickly if needed.
- **Old apps keep working.** The backend stays compatible with older app versions, and can require an update only when truly necessary.

RUNNING COSTS

What to budget for (indicative)

The figures below come from the providers' published pricing and third-party summaries. They change and vary by region, so please treat them as a starting point to be confirmed before commitment.

Item	Indicative cost	Notes
Google Play developer account	About USD 25, one time	A company account avoids the 14-day testing rule that applies to new personal accounts.
Apple Developer Program	About USD 99 per year	Needed to publish to the App Store or TestFlight. Requires a company identity number (D-U-N-S) for organisations.
Managed database	From roughly USD 15 per month at light use	Usage-based. A paid plan gives a longer restore window than a free tier.
Azure AI invoice reading	Pay per page	Controlled by the monthly page limits described above.
File storage and hosting	Usage-based	Depends on volume. We will size this once usage is known.
App store commission	15% to 30% on in-app digital sales	Only applies if subscriptions are sold inside the apps. Our recommendation is to sell plans on the web.

Distribution choice

Because this is a business tool, it can be distributed privately first (TestFlight for iPhone, an internal testing track for Android) before a public store listing. This reduces review friction while the apps mature.

RISKS AND DECISIONS

What we manage, and what we need from you

Main risks and how we handle them

Risk	How we reduce it
Changing a live system	Phase-by-phase changes, a test environment, a backup before every phase, and a rollback plan.
Data mixed between companies	Database-level protection plus automated tests that fail the build if separation breaks.
App store rejection	Native features (camera, secure sign-in), demo account for reviewers, plans sold on the web, and a store-safe white-label model (see the white-label section).
Unexpected AI cost	Page limits per project and company, duplicate-file detection, per-project keys.
Payment edge cases	Payment provider confirmations are the single source of truth; grace periods before restricting access.
Key person or secret loss	Encryption keys and backups documented, stored outside the code, and rehearsed.

Decisions we need to move forward

- **Payment provider** for subscriptions (for example Razorpay for India, Stripe for international customers).
- **Hosting** for the backend and database (Azure or another provider), and the region.
- **Roles** beyond Admin and Staff, such as accountant or read-only viewer.
- **Plan design**: price per project, billing period (monthly or yearly), the user limit per plan (and whether it adds up when more projects are bought), and the AI page allowance per project.
- **Support promise**: reply time per plan, support hours and languages.
- **Customer needs**: single sign-on or an integration API for larger customers.
- **Data promises**: retention periods and where customer data must be stored.

> **Next step:** a short workshop to confirm the plan design, hosting and payment provider. After that we can finalise the schedule and begin the company-separation work.